INFO8006 Introduction to Artificial Intelligence

Exercise session 7: Making decisions and reinforcement learning

Recap

Making decisions

Markov decision processes. A *Markov decision process* (MDP) is a tuple $(\mathcal{S}, \mathcal{A}, P, R)$ such that

- $\mathcal{S}$ is a set of *states* s;
- $\mathcal{A}$ is a set of *actions* a;
- P is a (stationary) *transition model* such that $P(s' \mid s, a)$ denotes the probability of reaching state s' if action a is done in state s;
- R is a *reward function* that maps immediate (finite) reward values $R(s)$ obtained in states s,

and $0 < \gamma \leq 1$ is the *discount factor*.

Utilities and policies. The *discounted utility* of a reward sequence is $V([r_0, r_1, r_2, \ldots]) = r_0 + \gamma r_1 + \gamma^2 r_2 + \ldots$, which is additive for $\gamma = 1$. A *policy* $\pi : \mathcal{S} \to \mathcal{A}$ maps states to actions, and the expected utility obtained by executing π starting in s is

$$V^\pi(s) = \mathbb{E}\left[\sum_{t \geq 0} \gamma^t R(s_t) \mid s_0 = s, \pi\right].$$

The *optimal policy* π^* maximizes the expected utility, and the *value* of a state is $V(s) = V^{\pi^*}(s)$. Then, $\pi^*(s) = \arg\max_a \sum_{s'} P(s' \mid s, a) V(s')$.

Bellman equation. The utility of a state is the immediate reward for that state, plus the expected discounted utility of the next state, assuming that the agent chooses the optimal action:

$$V(s) = R(s) + \gamma \max_a \sum_{s'} P(s' \mid s, a) V(s').$$

When the reward depends on the transition, $R(s, a, s')$, it moves under the sum: $V(s) = \max_a \sum_{s'} P(s' \mid s, a)[R(s, a, s') + \gamma V(s')]$.

Value iteration. Starting from initial estimates $V_0(s)$, *value iteration* repeats the *Bellman update*

$$V_{i+1}(s) = R(s) + \gamma \max_a \sum_{s'} P(s' \mid s, a) V_i(s')$$

until convergence. It converges to the unique solution of the Bellman equations whenever $\gamma < 1$, and also for $\gamma = 1$ when every policy reaches a terminal state.

Policy iteration. Policy iteration alternates *policy evaluation*, which calculates $V_i = V^{\pi_i}$ by solving the linear equations $V_i(s) = R(s) + \gamma \sum_{s'} P(s' \mid s, \pi_i(s)) V_i(s')$, and *policy improvement*,

$$\pi_{i+1}(s) = \arg\max_a \sum_{s'} P(s' \mid s, a) V_i(s').$$

Modified policy iteration replaces the exact evaluation by a few simplified Bellman updates $V_{i+1}(s) = R(s) + \gamma \sum_{s'} P(s' \mid s, \pi_i(s)) V_i(s')$.

Partially observable MDPs. When the state is not observed, the agent collects percepts e through a sensor model $P(e \mid s)$ and maintains a *belief state* b, a distribution over states. After doing a and perceiving e, it becomes

$$b' = \alpha \mathbf{P}(e \mid S') \sum_s \mathbf{P}(S' \mid s, a) b(s).$$

The optimal action depends only on b: a POMDP is an MDP over belief states, with the reward $\rho(b) = \sum_s b(s) R(s)$.

Reinforcement learning

Reinforcement learning. In *reinforcement learning*, the transition model $P(s' \mid s, a)$ and the reward function $R(s)$ are unknown: the agent learns from trials, whose trajectories are sequences of transitions (s, r, a, s'), each holding a state, its reward $r = R(s)$, the action done and the next state. In *passive RL*, the policy π is fixed and the agent learns the utilities $V^\pi(s)$; in *active RL*, it learns the optimal policy π^*. A *model-based* agent estimates $\hat{P}(s' \mid s, a)$ and $\hat{R}(s)$ from experience and solves the resulting empirical MDP. A *model-free* agent does not; for instance, *direct utility estimation* approximates $V^\pi(s)$ by the sample average of the observed rewards-to-go from s.

Temporal-difference learning. Temporal-difference (TD) *learning* updates $V^\pi(s)$ each time the agent experiences a transition $(s, r = R(s), a = \pi(s), s')$, to better agree with the Bellman equations for a fixed policy:

$$V^\pi(s) \leftarrow V^\pi(s) + \alpha(r + \gamma V^\pi(s') - V^\pi(s)),$$

where α is the *learning rate*.

Exploration. An agent that always acts greedily with respect to its current estimates may never discover better actions. An *ϵ-greedy* agent acts randomly with a small probability ϵ and follows its current policy otherwise. An *exploration function* $f(v, n)$, such as $f(v, n) = v + K/n$, instead makes the agent optimistic about the actions it has tried n times only, in $V^+_{i+1}(s) = R(s) + \gamma \max_a f\left(\sum_{s'} P(s' \mid s, a) V^+_i(s'), N(s, a)\right)$.

Q-Learning. The *state-action-value* $Q(s, a)$ is the expected utility starting out having taken action a from s and thereafter acting optimally, so that $V(s) = \max_a Q(s, a)$ and $\pi^*(s) = \arg\max_a Q(s, a)$. *Q-Learning* learns it in a model-free fashion, by updating $Q(s, a)$ each time the agent experiences a transition (s, r, a, s'):

$$Q(s, a) \leftarrow Q(s, a) + \alpha(r + \gamma \max_{a'} Q(s', a') - Q(s, a)).$$

It converges to the optimal Q-values, even when acting suboptimally, provided it explores enough. In *approximate Q-Learning*, $Q(s, a) = w_1 f_1(s, a) + \ldots + w_d f_d(s, a)$ is a linear model of features $f_k(s, a)$, and the update becomes $w_k \leftarrow w_k + \alpha(r + \gamma \max_{a'} Q(s', a') - Q(s, a)) f_k(s, a)$.

Exercises

In session: exercises 1, 4.

Making decisions

Exercise 1 Micro-Blackjack (CS188, Spring 2014)
A card-playing program plays Micro-Blackjack. It repeatedly draws a card (with replacement), equally likely to be a 2, 3 or 4, and adds its value to its score, which starts at 0. At each step, if the score is lower than 6, the program can either "draw" (d) or "cash" (c). Otherwise, it can only cash. When it cashes, the game stops and its utility is equal to its score (up to 5) plus 1, or zero if the score is 6 or higher. Until it cashes and after it, it receives no reward. There is no discount ($\gamma = 1$).

(a) Formalize Micro-Blackjack as an MDP.

(b) Derive the optimal policy for this MDP.

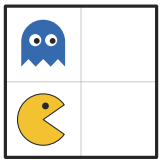
Exercise 2 Overheating machine
A maintenance robot looks after a machine that may overheat. It cannot measure the temperature directly: every hour, it only checks whether the machine's alarm is on, and decides whether to cool the machine for the next hour. The alarm is on with probability 0.8 when the machine is overheating, and with probability 0.1 when it is not. Cooling costs 5 (energy), whereas leaving an overheating machine without cooling for an hour costs 10 (wear). Between two checks, a machine that is not overheating starts overheating with probability 0.2, an overheating machine that is not cooled keeps overheating, and a machine that has been cooled is no longer overheating at the next check. When the robot starts, the machine is overheating with probability 0.5.

(a) Draw the dynamic decision network of this problem, as well as its conditional probability tables and its reward function.

(b) Give the sequence of belief states if the robot observes the alarm and acts as follows: $(e_t, f_t) = (0,0), (1,1), (0,0), (0,0)$ for $t = 0, 1, 2, 3$.

(c) From the sequence of belief states, determine the sequence of expected rewards.

Exercise 3 Pursuit evasion (CS188, Spring 2014)
Pacman is trapped in the 2 by 2 maze below with a hungry ghost. When it is its turn to move, Pacman must move one step horizontally or vertically to a neighboring square. When it is the ghost's turn, it must also move one step horizontally or vertically. The ghost and Pacman alternate moves. After every move (by either the ghost or Pacman), if Pacman and the ghost occupy the same square, Pacman is eaten and receives a utility of -100. Otherwise, it receives a utility of 1. The ghost attempts to minimize the utility that Pacman receives. Pacman makes the first move. The game is not guaranteed to terminate.



For example, with a discount factor of $\gamma = 1$, if Pacman moves right, ghost moves down, then Pacman moves left, Pacman earns a reward of 1 initially ($R(s_0)$) and after the first two moves and -100 after the last move, for a total utility of -97.

(a) Assume a discount factor $\gamma = 0.5$, where the discount factor is applied once every time either Pacman or the ghost moves. What is the Minimax value of the complete (infinite) game?

(b) Why are the value/policy iteration algorithms superior to Minimax for solving this game?

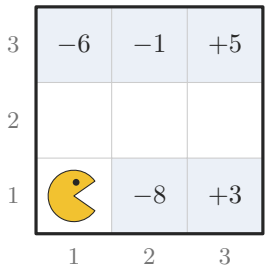
This game is similar to an MDP because rewards are earned at every timestep. However, it is also an adversarial game involving decisions by two agents. Let s be the state (*e.g.* the positions of Pacman and the ghost and who is playing) and let $\mathcal{A}(s)$ be the set of actions available to the player in state s. Let $s' = T(s, a)$ denote the successor of a state s resulting from the action a. Finally, let $R(s)$ denote the utility received after moving to state s.

(c) Write down an expression (analogous to Bellman equation) for $V(s)$, the value of the current state if it is Pacman's turn.

Reinforcement learning

Exercise 4 Passive reinforcement learning

Consider the grid-world below and an agent that follows a fixed, possibly stochastic, policy π, whose expected utilities V^π it learns (passive reinforcement learning). The agent starts from the bottom-left corner and can take the actions `north` (N), `south` (S), `west` (W) and `east` (E). Rewards are only awarded for reaching the terminal (shaded) states.



You observe the following trials, whose trajectories are sequences of tuples $(s_t^i, r_t^i, a_t^i, s_{t+1}^i)$.

t	Trial 1	Trial 2	Trial 3	Trial 4	Trial 5
0	$(1,1), 0, N, (1,2)$	$(1,1), 0, N, (1,2)$	$(1,1), 0, N, (1,2)$	$(1,1), 0, N, (1,2)$	$(1,1), 0, N, (1,2)$
1	$(1,2), 0, E, (2,2)$	$(1,2), 0, E, (2,2)$	$(1,2), 0, E, (2,2)$	$(1,2), 0, E, (2,2)$	$(1,2), 0, E, (2,2)$
2	$(2,2), 0, N, (2,3)$	$(2,2), 0, E, (3,2)$	$(2,2), 0, S, (2,1)$	$(2,2), 0, E, (3,2)$	$(2,2), 0, E, (3,2)$
3	$(2,3), -1, \varnothing, \varnothing$	$(3,2), 0, N, (3,3)$	$(2,1), -8, \varnothing, \varnothing$	$(3,2), 0, W, (2,2)$	$(3,2), 0, S, (3,1)$
4		$(3,3), +5, \varnothing, \varnothing$		$(2,2), 0, N, (2,3)$	$(3,1), +3, \varnothing, \varnothing$
5				$(2,3), -1, \varnothing, \varnothing$	

Assuming a discount factor $\gamma = 1$,

(a) Perform direct utility estimation of the expected utilities $V^\pi(s)$, given the first four trials, averaging the rewards-to-go observed at every visit of each state.

(b) Update the estimated expected utilities with respect to the fifth trial using temporal-difference learning. Assume a learning rate $\alpha = 0.5$.

Exercise 5 Q-Learning

An agent is in an unknown environment where there are three states $\{A, B, C\}$ and two actions $\{0, 1\}$. We are given the following transitions, generated by taking actions in the environment. Here, the reward depends on the transition, $r = R(s, a, s')$, so that each tuple (s, a, r, s') lists the state, the action, the reward received for the transition and the next state.

s	a	r	s'
A	0	$+2$	A
C	1	-2	A
B	1	$+1$	B
A	0	-1	B
B	1	-2	C
C	0	$+4$	B
B	0	$+1$	A

Assuming a discount factor $\gamma = 0.5$ and a learning rate $\alpha = 0.75$,

(a) Apply the Q-Learning algorithm to obtain state-action-value $Q(s, a)$ estimates, processing the transitions once, in order. Estimates are initialized to 0.

(b) We now switch to a feature-based estimator $Q(s, a) = w_0 + w_1 f_1(s, a)$, with $f_1(s, a) = 2a - 1$. Starting from weights $w_0 = w_1 = 0$, update the weights according to the approximate Q-Learning algorithm, processing the transitions once, in order.

(c) Why can this estimator not represent the Q-values of (a)?

Exercise 6 Robot football

ULiège's robot football team plays against UCL's robots in a RoboCup match next week. After a lot of losses this season, the team needs a better attack strategy, and its coaching program models the attack as a Markov decision process. It considers four states `close` (`C`), `away` (`A`), `fail` (`F`), and `goal` (`G`), the last two being terminal, and two actions `pass` (`P`) and `shoot` (`S`). Although the transition probabilities are unknown, the possible transitions (s, a, s') are known, and to each transition is associated a reward $R(s, a, s')$.

s	a	s'	$R(s, a, s')$
C	P	C	$+1$
C	P	A	-1
C	P	F	-2
C	S	C	$+3$
C	S	F	-5
C	S	G	$+10$
A	P	C	$+2$
A	P	A	0
A	P	F	-3
A	S	C	$+3$
A	S	F	-10
A	S	G	$+20$

The current strategy of the team is to always shoot. Last match, the robots had several attack opportunities, resulting in the following transitions.

s	C	C	C	C	A	A	A	A
a	S	S	S	S	S	S	S	S
s'	G	C	G	F	F	C	F	F

Assuming a discount factor $\gamma = 0.75$ and a learning rate $\alpha = 0.25$,

(a) Build an estimator of the transition model $P(s' \mid s, a)$ and, from it, determine the expected utility V^π of the team's current policy π.

The coaching program finds the logs of the previous season, when the team had much more success, and selects the following instructive transitions.

s	C	C	A	A	C	A	A	C
a	S	S	S	P	P	P	S	P
s'	G	C	F	C	C	C	F	F

(b) Given the selected transitions, apply the Q-Learning algorithm to obtain state-action-value $Q(s, a)$ estimates, processing the transitions once, in order. Estimates are initialized to 0.

(c) Determine the optimal policy according to the state-action-value estimates.

(d) None of the selected transitions is a goal scored from away, $(\mathtt{A}, \mathtt{S}, \mathtt{G})$, worth +20. Should the team trust the policy of (c) in state A? How could the coaching program learn more?

Exercise 7 Contraction of the Bellman update (AIMA ch. 17)

Let B denote the Bellman update, which maps a utility function V to the function

$$(BV)(s) = R(s) + \gamma \max_{a \in \mathcal{A}(s)} \sum_{s'} P(s' \mid s, a) V(s').$$

The Bellman update is a contraction, which guarantees the convergence of value iteration for $\gamma < 1$.

(a) Show that, for any two functions $f : \mathcal{X} \to \mathbb{R}$ and $g : \mathcal{X} \to \mathbb{R}$ over a finite set $\mathcal{X}$,

$$|\max_x f(x) - \max_x g(x)| \leq \max_x |f(x) - g(x)|.$$

(b) Apply the previous result to prove that the Bellman update is a contraction of factor γ, that is, for any two utility functions $V, V' : \mathcal{S} \to \mathbb{R}$,

$$\|BV - BV'\|_\infty \leq \gamma \|V - V'\|_\infty,$$

where $\|V\|_\infty = \max_s |V(s)|$.

Exercise 8 Value iteration (AIMA ch. 17)

Consider the 3×3 world below, where each square is labelled with its reward. The upper right square is a terminal state. In each state, the agent can move north, south, east or west: 80% of the time it goes in the direction it selects, and the rest of the time it moves at right angles to the intended direction (10% on each side). A move into a wall leaves the agent where it is.

r	-1	$+10$
-1	-1	-1
-1	-1	-1

Apply value iteration for this world for each value of r below, with a discount factor $\gamma = 0.99$. Show the policy obtained in each case. Explain intuitively why the value of r leads to each policy.

(a) $r = -100$

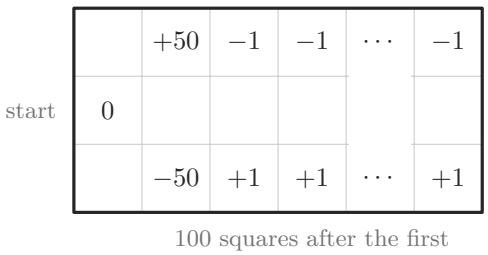
(b) $r = -3$

(c) $r = 0$

(d) $r = +3$

Exercise 9 Discount factor (AIMA ch. 17)

Consider the 101×3 world below. In the start state, of reward 0, the agent has a choice of two deterministic actions, `up` or `down`, but in the other states the agent has one deterministic action, `right`, until it leaves the world at the right end. Assuming a discounted reward function, compute the utility of each action as a function of γ. For what values of the discount factor γ should the agent choose `up` and for which `down`?



Exercise 10 Policy iteration (AIMA ch. 17)

Consider an undiscounted MDP having three states, $(1, 2, 3)$, with rewards -1, -2, 0, respectively. State 3 is a terminal state. In states 1 and 2 there are two possible actions: a and b. The transition model is as follows:

- In state 1, action a moves the agent to state 2 with probability 0.8 and makes the agent stay put with probability 0.2.
- In state 2, action a moves the agent to state 1 with probability 0.8 and makes the agent stay put with probability 0.2.
- In either state 1 or state 2, action b moves the agent to state 3 with probability 0.1 and makes the agent stay put with probability 0.9.

Answer the following questions:

(a) What can be determined *qualitatively* about the optimal policy in states 1 and 2?

(b) Apply policy iteration, showing each step in full, to determine the optimal policy and the values of states 1 and 2. Assume that the initial policy has action b in both states.

(c) What happens to policy iteration if the initial policy has action a in both states? Does discounting help? Does the optimal policy depend on the discount factor?

This simple example actually reflects many real-world situations in which one must weigh the value of an immediate action versus the potential long-term consequences, such as choosing to dump pollutants into a lake.

Quiz

1. Which of the following is true? In Markov Decision Processes, ...
   - ◯ the closer the discount factor γ to 0, the higher the utility of future rewards.
   - ◯ the closer the discount factor γ to 0, the longer Value Iteration may take to converge.
   - ◯ the closer the discount factor γ to 1, the greedier the optimal agent.
   - ◯ the closer the discount factor γ to 1, the longer Value Iteration may take to converge.
2. Q-Learning ...
   - ◯ is a model-based reinforcement learning algorithm.
   - ◯ is an on-policy reinforcement learning algorithm.
   - ◯ converges to an optimal policy, but only when acting optimally.
   - ◯ requires enough exploration (*e.g.* ϵ-greedy) to converge to an optimal policy.
3. Assume that we run ϵ-greedy Q-Learning until convergence. What is the optimal policy π^* we obtain for an arbitrary state s?
   - ◯ $\pi^*(s) = \arg\max_s V(s)$
   - ◯ $\pi^*(s) = \begin{cases} \arg\max_a Q(s,a) & \text{with probability } 1-\epsilon \\ \text{random action} & \text{with probability } \epsilon \end{cases}$
   - ◯ $\pi^*(s) = \arg\max_a Q(s,a)$
   - ◯ $\pi^*(s) = \arg\max_s Q(s,a)$
4. Let us consider a robot wandering around at the Montefiore Institute. Which of the following is true?
   - ◯ The robot and its environment can be modeled as a partially observable MDP.
   - ◯ The Kalman filter can be used for determining accurately its past trajectory in the building.
   - ◯ A convolutional neural network would be too large to fit in the robot's memory.
   - ◯ Value iteration (with an admissible heuristic) can be used for decoding the speech of its visitors.
5. The Bellman equations $Q(s,a) = R(s) + \gamma \sum_{s'} P(s' \mid s,a) \max_{a'} Q(s',a')$ form a system of n non-linear equations with as many unknowns. Which of the following is true?
   - ◯ n is the size of the action space $\mathcal{A}$.
   - ◯ n is the size of the state space $\mathcal{S}$.
   - ◯ n is the size of the state-action space $\mathcal{S} \times \mathcal{A}$.
   - ◯ n is the sum of the sizes of the action and state spaces.
6. In DQN (Mnih et al., 2015), a reinforcement learning agent is trained ...
   - ◯ to classify images.
   - ◯ to control a robot in a simulated environment.
   - ◯ to play Atari video games.
   - ◯ to play the game of Go.
7. In DQN (Mnih et al., 2015), the Q-table is approximated with ...
   - ◯ a hash table.
   - ◯ a transposition table.
   - ◯ a linear regression model.
   - ◯ a convolutional neural network.